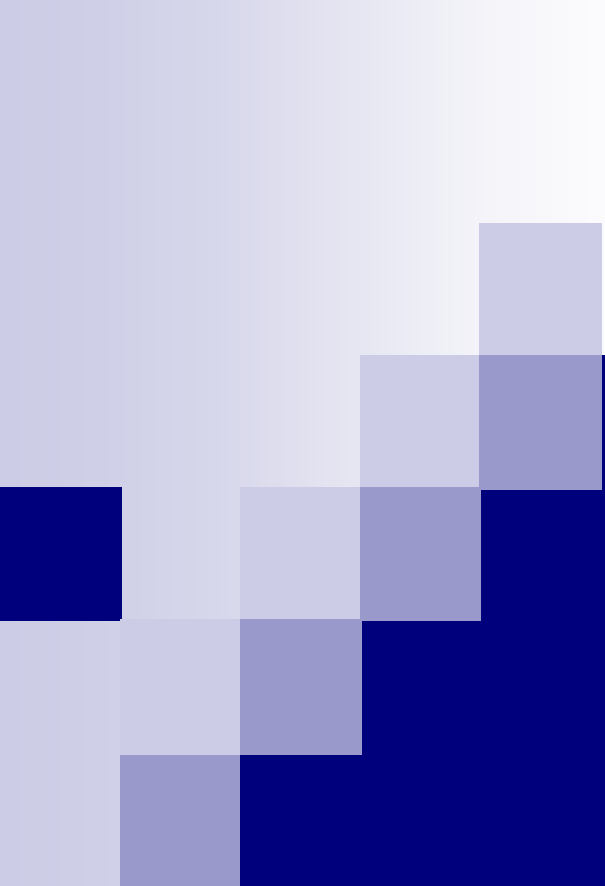




Quality Design Concepts – I

Lecture # 11

Recap

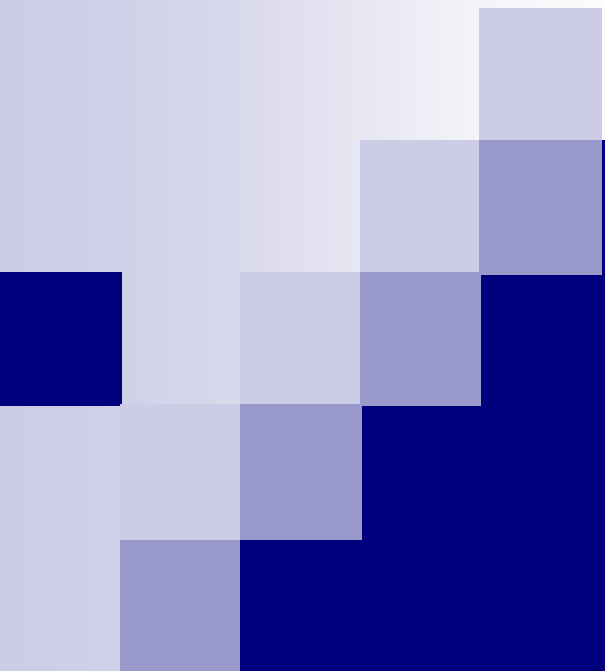
- Introduced software design and how it relates with software quality assurance



Today's Lecture

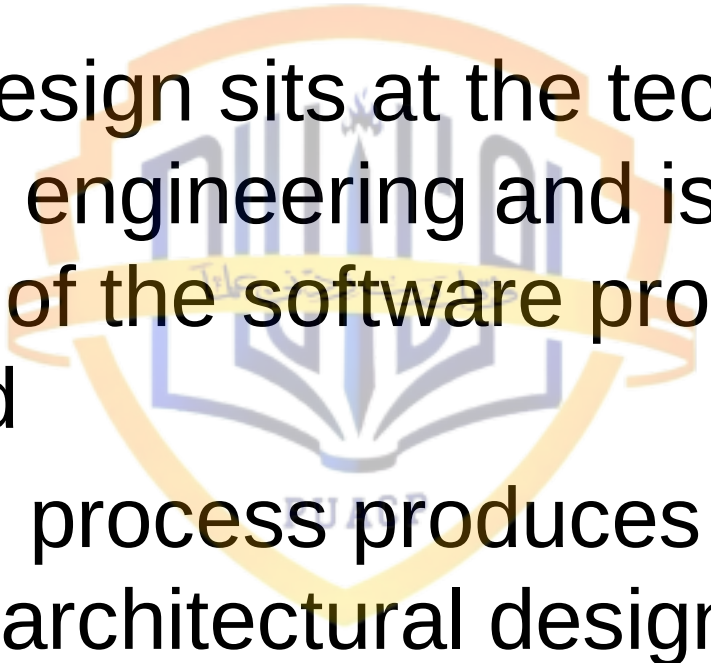
- Design process and software quality assurance
- Design model and software quality assurance





Design Process



- 
- 
- Software design sits at the technical kernel of software engineering and is applied regardless of the software process model that is used
 - The design process produces a data design, an architectural design, an interface design, and a component design

Data Design

- The data design transforms the information domain model created during analysis into the data structures that will be required to implement the software

Architectural Design

- It defines the relationship between major structural elements of the software. Architectural design representation is derived from system specification, analysis model, and interaction of subsystems

Interface Design

- Interface design describes how the software communicates within itself, with systems that interoperate with it, and with humans who use it. An interface implies a flow of information

Component Design

- Component-level design transforms structural elements of software architecture into a procedural description of software components

Think the Right Way

- To achieve a good design, people have to think the right way about how to conduct the design activity
 - Katharine Whitehead
- Let' now discuss the design principles, which help us to follow the design process accurately

Design Process Principles - 1

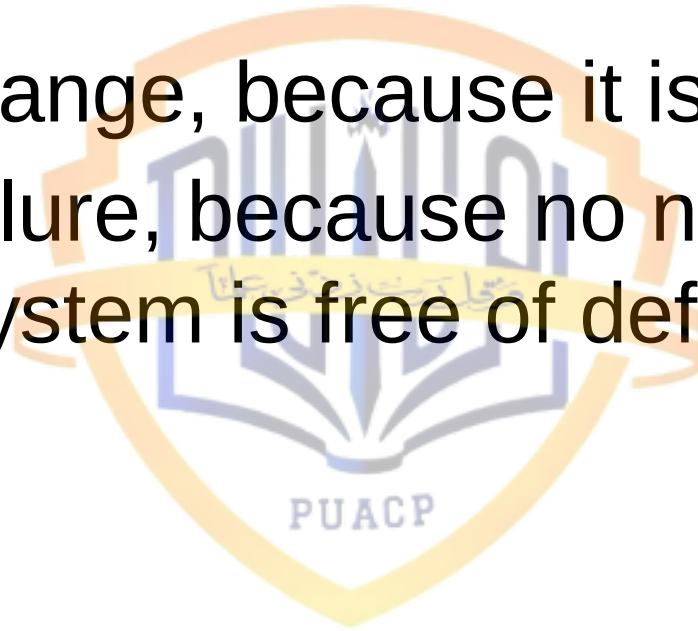
- The design process should not suffer from “tunnel vision”
- The design should not reinvent the wheel
- The design should “minimize the intellectual” distance between the software and the problem as it exists in the real world

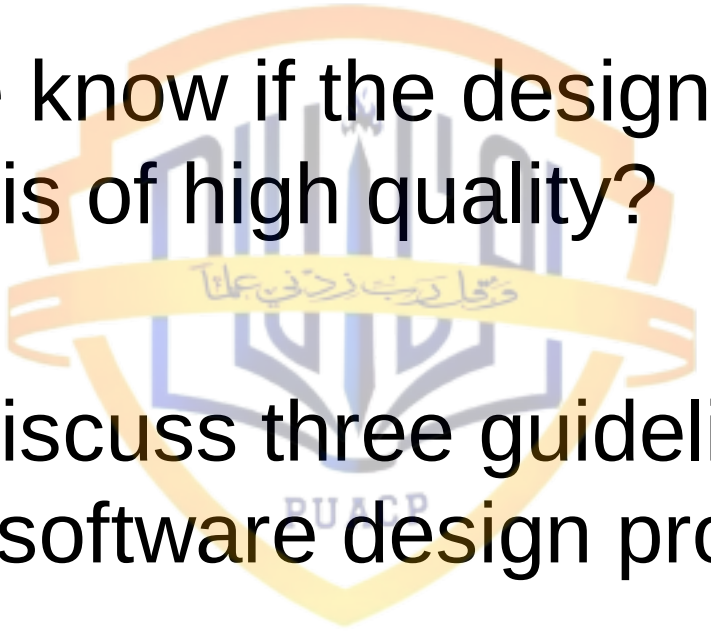
Design Process Principles - 2

- The design should exhibit uniformity and integration
- The design should be assessed for quality as it is being created, not after the fact
- The design should be reviewed to minimize conceptual (semantic) errors

When Applying them

- Plan for change, because it is inevitable
- Plan for failure, because no nontrivial software system is free of defects



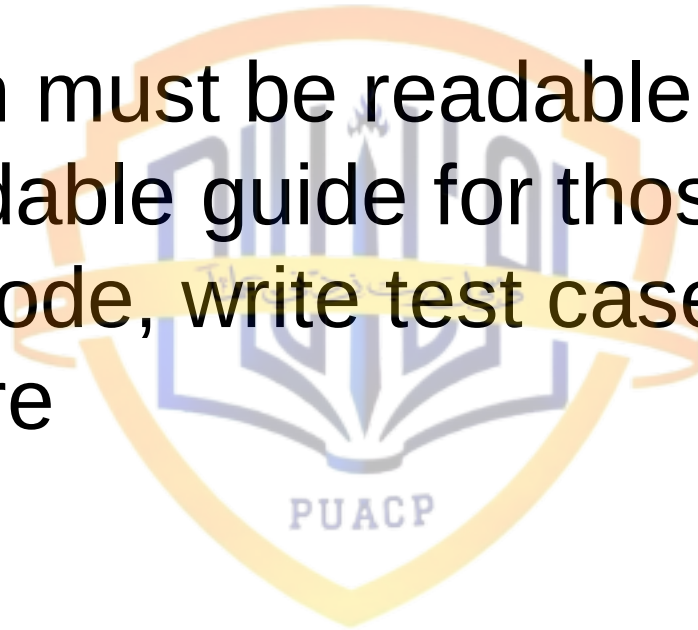
- 
- 
- How do we know if the design we have developed is of high quality?
 - Now let's discuss three guidelines for evaluating software design process

Design Process Evaluation Guide #1

- The design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer

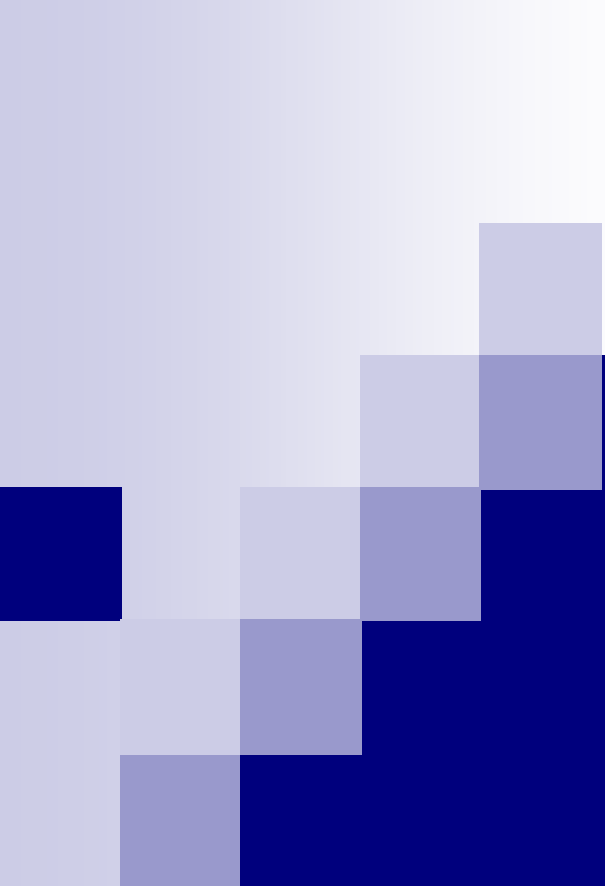
Design Process Evaluation Guide # 2

- The design must be readable and understandable guide for those who generate code, write test cases, and test the software



Design Process Evaluation Guide #3

- The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective



Design Model/Representation



Design Model Principles - 1

- Separation of concerns
- Modeling real-world objects
- Minimizing the interactions among cohesive design components
- The design should be traceable to the analysis model

Design Model Principles - 2

- The design should be structured to accommodate change
- The design should be structured to degrade gently, even when aberrant data, events, or operating conditions are encountered
- Design is not coding, coding is not design

- Let's look at some design model guidelines, before we discuss the design quality attributes



Guidelines for Good Design Model - 1

- A design should exhibit an architectural structure that
 - Has been created using recognizable design patterns
 - Is composed of components that exhibit good design characteristics
 - Can be implemented in an evolutionary fashion, facilitating implementation and testing

Guidelines for Good Design Model - 2

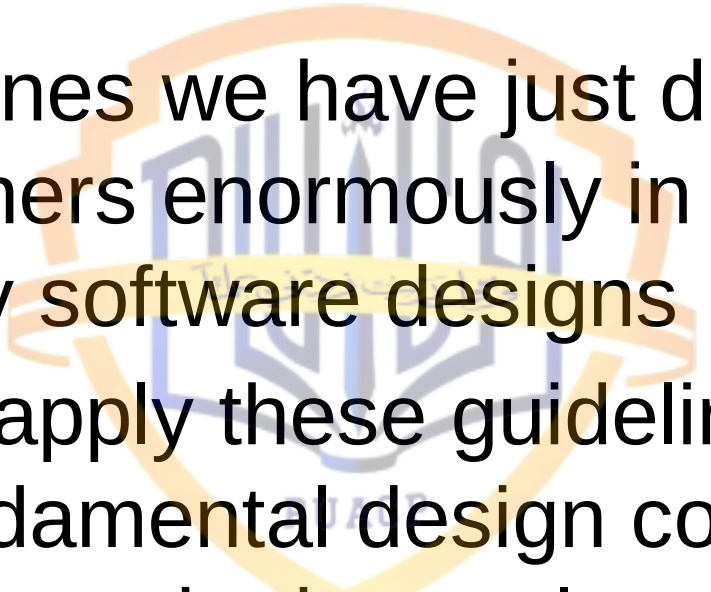
- A design should be modular; that is software should be logically partitioned into elements that perform specific functions and sub-functions
- The design should contain distinct representations of data, architecture, interfaces, and components (modules)

Guidelines for Good Design Model - 3

- A design should lead to data structures that are appropriate for the objects to be implemented and are drawn from recognizable data patterns
- A design should lead to components that exhibit independent functional characteristics

Guidelines for Good Design Model - 4

- A design should lead to interfaces that reduce the complexity of connections between modules and with external environment
- A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis

- 
- 
- The guidelines we have just discussed help designers enormously in developing high quality software designs
 - Designers apply these guidelines with the help of fundamental design concepts, which improve the internal and external quality of software design

Questions Answered by Design Concepts

- What criteria can be used to partition software into individual components?
- How is function or data structure detail separated from a conceptual representation of software?
- What uniform criteria define the technical quality of a software design?

Quality Design Concepts

- The beginning of wisdom for a software engineer is to recognize the difference between getting a program to work, and getting it right
 - M. A. Jackson
- These design concepts provide the necessary framework for “getting it right” or to produce highest possible software designs

Abstraction

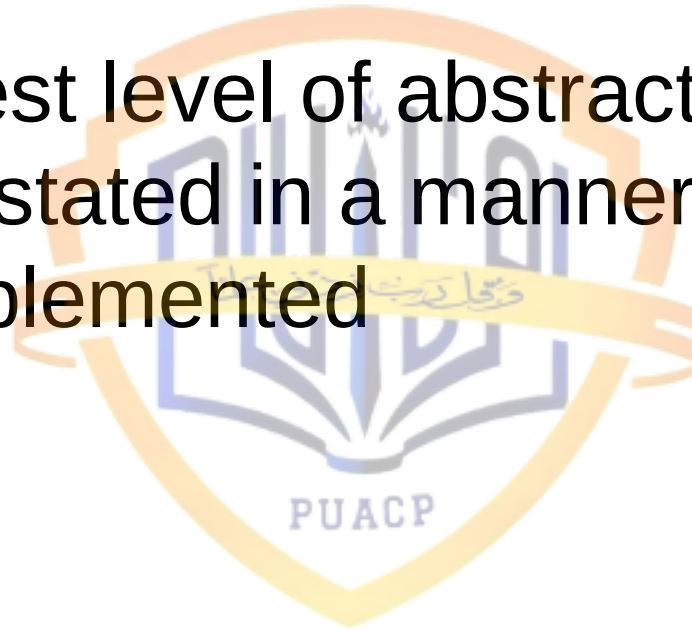
- Abstraction permits one to concentrate on a problem at some level of generalization without regard to irrelevant low-level details
- Abstraction is one of the fundamental ways that we as humans cope with complexity
 - Grady Booch

Levels of Abstraction - 1

- At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment
- At lower levels of abstraction, a more procedural orientation is taken. Problem-oriented terminology is coupled with implementation-oriented terminology in an effort to state a solution

Levels of Abstraction - 2

- At the lowest level of abstraction, the solution is stated in a manner that can be directly implemented



Types of Abstraction - 1

■ Procedural abstraction

- Named sequence of instructions that has a specific and limited function
- Example: Open door

■ Data abstraction

- Named collection of data that describes a data object
- Example: any object (ADT)

Types of Abstraction - 2

- Control abstraction

- ☐ Implies a program control mechanism without specifying internal details
- ☐ Example: synchronization semaphore

Refinement - 1

- A program is developed by successively refining levels of procedural detail
- A hierarchy is developed decomposing a macroscopic statement of function in a step-wise fashion until programming language statements are reached
- Refinement is actually a process of elaboration

Refinement - 2

- There is a tendency to move immediately to full detail, skipping the refinement steps. This leads to errors and omissions and makes the design much more difficult to review. Perform stepwise refinement
- Abstraction and refinement are complementary concepts

Modularity - 1

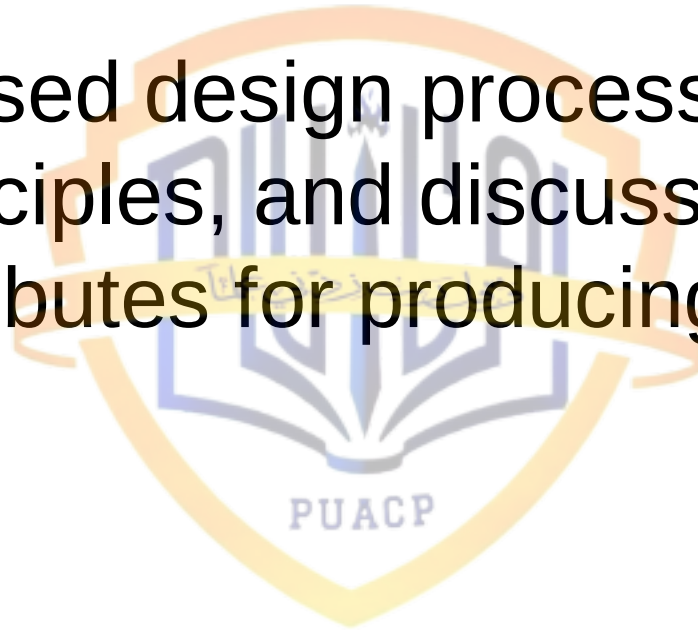
- One of the oldest concepts in software design
- Software is divided into separately named and addressable components, often called, modules, that are integrated to satisfy problem requirements

Modularity - 2

- Modularity is the single attribute of software that allows a program to be intellectually manageable
- Don't over modularize. The simplicity of each module will be overshadowed by the complexity of integration

Summary

- We discussed design process and design model principles, and discussed the some quality attributes for producing high quality design



References

- Software Engineering: A Practitioner's Approach by Roger Pressman (Chapter 13)

